

Building zorp

A human-checkpointed research agent over a tamper-evident evidence record

Aviskaar · zorp v0.3.2 · August 2026

The problem

A large language model will produce a fluent answer to a hard question in seconds. What it will not do is tell you whether to believe it, what evidence it weighed, or what pointed the other way. The ambitious response has been to automate the entire research loop end to end and append review afterward. That fails at the point of consumption: a manuscript an AI wrote end to end is not submittable at most venues, and a technical recommendation nobody can vouch for is not actionable however sound its reasoning. Authorship and accountability cannot be retrofitted onto a finished artifact.

zorp starts from the opposite premise. **The human is the author of record**, and the system’s job is to make that person’s evidence trail complete, inspectable, and hard to revise after the fact.

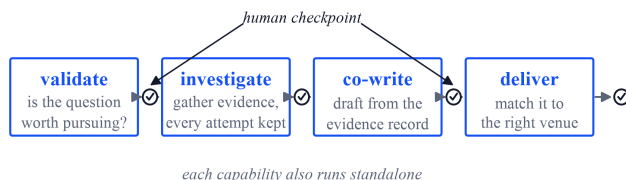
Three commitments

1. **The record is the product.** Every attempt is stored — failures and contradictory results included — as typed rows rather than narrative logs, so a claim resolves to a measurement instead of a paragraph the agent wrote about itself.
2. **Falsification is committed in advance.** Before any evidence is gathered, the hypothesis, metric, and a numeric **kill threshold** — supplied by a human, never proposed by the agent — are written to a git-committed **prereg.md** and hash-verified on every load.
3. **Capabilities are standalone and checkpointed.** Four capabilities run alone and chain only through explicit human checkpoints. A checkpoint reached with nobody available is an error, not a skipped step; unattended runs must opt in visibly.

Architecture

zorp is a single Rust workspace compiled to one multi-subcommand binary with ten crate members: **zorp** core (model transport), **zorp-agent** (tools, sandboxing, verification, sessions, MCP), **zorp-mcp**, **zorp-eval**, **zorp-track** (the evidence foundation), plus **zorp-web**, **zorp-search**, **zorp-skill**, **zorp-recall**, and **erbga**, a genetic algorithm for graph community detection. The execution layer is an in-tree fork of **quecto**, extended rather than depended upon.

Two rules follow from the commitments. **The foundation does not know its consumers:** **zorp-track** owns tracks, evidence records, pre-registration, and checkpoints, and knows nothing of any capability, so tamper evidence is testable apart from capability logic. **Heavy dependencies are opt-in:** DuckDB and LanceDB sit behind the **research** feature, so a base-agent user never pays their cold-cache build. Parallel workers are additional copies of the same binary run as subprocesses, not a second program.



The four capabilities and the checkpoints between them. Each also runs alone; validate and deliver refuse to run without a configured tool. Figure reused from [docs/paper/figures](#).

Mechanisms that carry the guarantee

Tamper evidence. On every track load, **zorp-track** re-hashes each **prereg.md** (SHA-256 over raw bytes) against the hash recorded at commit time; mismatch or absence refuses the run outright. DuckDB/LanceDB stores are regenerable indexes over these files — the committed files are the source of truth, and a commit timestamp cannot be moved after results are seen.

The evidence record. Six DuckDB tables: **tracks**, **preregistrations**, **experiments**, **metrics**, **checkpoints**, **validations**. A metric is a (key, value_type, value) tuple in typed columns, so drafting is a lookup, not a paraphrase: “p99 latency fell 40%” must resolve to a row someone who was absent can check.

Checkpoints. Two modes only: **Interactive** (default) and **AutoApprove** (explicit opt-in). There is no third mode. Each records what the human was shown and what they decided.

Critique. A gate between drafting and delivery, not a capability: it audits the draft against the track’s own record *in code* — arithmetic, not asking a model whether it likes its draft — flags figures the record cannot account for, revises within a bound you set, and cannot move the kill threshold.

Tool gating. `validate` fails fast without a search-capable tool rather than score feasibility on no evidence; `deliver` requires a venue tool. A tool searching your own saved notes does not count as external search.

Capability	What it does	Needs
<code>validate</code>	Scores redundancy and feasibility; every score carries a citation	Search-capable MCP tool
<code>investigate</code>	One pre-registered attempt per invocation; every attempt enters the record	None
<code>co-write</code>	Drafts <code>draft.md</code> from the record alone; human stays author of record	None
<code>deliver</code>	Matches the draft against real venues; writes a ranked shortlist	Venue-database tool

Since the paper

The decision log (through 2026-08-22) records: **aryabhata**, an anomaly-driven discovery layer inside **zorp-track** (record-and-readers, deliberately not a fifth capability), with **erbga** wired in above the exact-solver crossover; opt-in **forecasting** feeding a calibration harness whose go/no-go verdict is computed arithmetic, left unenforced for people; person-launched **review panels** (no model-callable spawn, read-only reviewers); and loopback-only conversation **memory** whose recalled text arrives as fenced quotation that grants nothing. Security posture is uniform throughout: egress is opt-in per feature, skills add guidance never permissions, auto-approve cannot pass a denylisted command, and the web server binds loopback unless tokenized.

Status

Metric	Value	Source
At paper snapshot	538 passing tests / 24,965 LOC	Commit <code>fd07e81</code> , 2026-08-13
Working tree today	68,976 LOC Rust · 1,404 <code>#[test]</code> sites	Measured 2026-08-22
History & packaging	189 commits since 2026-08-08 · v0.3.2 · MSRV 1.95	<code>git</code> ; <code>Cargo.toml</code>

Tests cover the failure modes the design turns on — tampered pre-registration hashes, checkpoints with no terminal and no flag, capabilities invoked without their required tool, index rebuild after database deletion — plus integration tests driving a stub MCP server over stdio. All four capabilities were built in under a week: read this as a design report, not a mature-system retrospective.

What is *not* claimed

- **No comparative evaluation yet.** Tests establish mechanisms behave as specified — tampering detected, gates refusing, records surviving index loss — not that investigations are good.
- **Tamper evidence is scoped.** It proves the committed threshold was not altered post hoc; it inherits git’s trust model (history rewriting defeats it) — a defense against quiet drift, not adversaries.
- **Deferred:** LanceDB provisioned but unused; one active track per session; deliver scoped to academic venues though the design need not be.
- **Throughput bounded by design** — human checkpoints make zorp a poor fit where volume matters more than defensibility.

Roadmap and availability

Next: a published start-to-finish investigation trace; a grounded-versus-baseline evaluation reported including where zorp loses; systems paper submission (complete draft at `docs/paper/`). MIT licensed; github.com/aviskaar/zorp; prebuilt Linux/macOS binaries; ~150 MB non-root container image.

Sources: `README.md` · `docs/paper/zorp-paper.md` · `docs/DECISIONS.md` (through 2026-08-22) · `Cargo.toml` · working-tree measurements of 2026-08-22.